



程序设计基础及语言 —— C++ 大学教程

第3章 类、对象和字符串的介绍

邱远

东南大学 网络空间安全学院
yuanqiu@seu.edu.cn

2026 年 10 月



目录

面向对象编程

类的定义与使用

成员函数

数据成员

构造函数

多文件





为什么需要类和对象

- 基本数据类型 `bool`, `char`, `short`, `int`, `long`, `float`, `double`

- 现在需要记录一位同学的姓名¹、一卡通号、成绩

```
std::string name = "张三";  
int id = 114514;  
double grade = 4.0;
```

- 还有另一位同学，我们只得重新起变量名

```
std::string name2 = "李四";  
int id2 = 114515;  
double grade2 = 3.9;
```

- 如果要储存全班 30 个同学的信息，需要 90 个变量
- 我们后续会学到，可以创建三个数组

```
std::string name[30] = {"张三", "李四"};  
int id[30] = {114514, 114515};  
double grade[30] = {4.0, 3.9};
```

- 这样依然很不方便，三组数据必须始终保持相同顺序，否则很容易对应错学生（如误插入、误删除）

¹我们很快会学到用 `std::string` 存字符串



类和对象

这些变量有何特点？

- 每一个学生都有name, id, grade
- 同一个学生的变量关系很密切
- 我们希望有一个新的“类型”用来存放学生的数据

```
class Student { };
```

- 这个类型的每个“变量”对应一个学生
Student bob;

类 (Class)

用户自定义的**数据类型**，描述具有共同属性(数据/Attribute) 或行为(方法/Method) 的一类事物

对象 (Object)

根据某个类创建出的**具体实例**



理解类和对象

类中不仅可以定义数据（数据成员），还可以定义方法（成员函数）

类	数据成员	成员函数	对象
Student	name, id, grade	study, sleep	alice, bob
Cat	age, weight	walk, meow	kitty, luna
Dog	age, weight	walk, bark	shepherd, husky
Window	is_active, is_fullscreen	close, resize	chrome, terminal
Sun	temp, radius	rise	the_sun

表 1: 类和对象的对比

类描述“一类事物”，对象表示其中“一个具体事物”



目录

面向对象编程

类的定义与使用

成员函数

数据成员

构造函数

多文件





程序一：具有成员函数的类

```
1  /** @file fig03_01.cpp
2   ** Define a class with a member function, create an object that calls it
3   */
4  #include <iostream>
5
6  /** @class GradeBook
7   ** @brief First definition of class */
8  class GradeBook {
9  public:
10     /** @brief Display Welcome message */
11     void displayMessage() const {
12         std::cout << "Welcome to the Grade Book!" << std::endl;
13     } // end function
14 }; // end class
15
16 int main() {
17     GradeBook myGradeBook; // create a GradeBook object named myGradeBook
18     myGradeBook.displayMessage(); // call object's member function
19 }
```



Line 6-14: 类的定义

<https://cplusplus.com/doc/tutorial/classes/>

```
6  /** @class GradeBook
7      ** @brief First definition of class */
8  class GradeBook {
9  public:
10     /** @brief Display Welcome message */
11     void displayMessage() const {
12         std::cout << "Welcome to the Grade Book!" << std::endl;
13     } // end function
14 }; // end class
```

C++ 类

```
class 类名 {
public:
    变量与函数定义;
protected:
    变量与函数定义;
private:
    变量及函数定义;
};
```



Line 6-14: 类的定义

<https://cplusplus.com/doc/tutorial/classes/>

```
6  /** @class GradeBook
7      ** @brief First definition of class */
8  class GradeBook {
9  public:
10     /** @brief Display Welcome message */
11     void displayMessage() const {
12         std::cout << "Welcome to the Grade Book!" << std::endl;
13     } // end function
14 }; // end class
```

- **class**关键字用于声明类
- 类名一般采用 Pascal 风格：大写字母开头，后续采用驼峰命名法
- 类体 (body) 用大括号 {} 括起，分号结尾（分号不能少）
- **public**, **protected**, **private**关键字：访问说明符
 - **public**：谁都能看
 - **private**：只有自己能看
 - **protected**：自己和朋友能看
 - **class**中不加说明的，默认为**private**



Line 10-12: 成员函数的定义

https://en.cppreference.com/cpp/language/member_functions

```
6  /** @class GradeBook
7   ** @brief First definition of class */
8  class GradeBook {
9  public:
10     /** @brief Display Welcome message */
11     void displayMessage() const {
12         std::cout << "Welcome to the Grade Book!" << std::endl;
13     } // end function
14 }; // end class
```

回顾：C++ 函数

返回类型 函数名(输入参数) { 函数体; }

- 类中的函数称为**成员函数** (member function)
- **void**表示函数没有返回值，因此可以没有**return**
- 跟main函数一样，空的圆括号代表函数没有入参
- 我们晚些解释**const**在此处的含义



Line 17-18: 构建对象与函数调用

```
16 int main() {  
17     GradeBook myGradeBook; // create a GradeBook object named myGradeBook  
18     myGradeBook.displayMessage(); // call object's member function  
19 }
```

C++ 构建对象

类名 对象名;

C++ 调用成员函数

对象名. 函数名 (输入参数);

- 把类理解为特殊的类型，定义对象和定义基础类型的变量一样
 - C++ 的可扩展性之一就是运行程序员定义新的类型
- 点运算符 (.) 用于调用对象的成员 (变量与函数)，理解为“的”
 - 优先级非常高，和圆括号一个级别



对比：全局函数调用与成员函数调用

```
// example
#include <iostream>

void showMessage() {
    std::cout << "This is a non-member function." << std::endl;
}

class GradeBook {
public:
    void displayMessage() const {
        std::cout << "Welcome to the Grade Book!" << std::endl;
    }
};

int main() {
    showMessage();
    GradeBook myGradeBook;
    myGradeBook.displayMessage();
}
```

推荐的格式

```
int main() { // 左大括号空格不换行
    return 0; // 大括号内缩进一个 4 空格
}           // 右大括号换行，并与左大括号所在行对齐
```

UML 类图

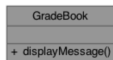
统一建模语言 (Unified Modeling Language)

可视化面向对象模型，从上到下依次为类名、数据成员、成员函数

GradeBook Class Reference

First definition of class. [More...](#)

Collaboration diagram for GradeBook:



Public Member Functions

```
void displayMessage () const
```

Display Welcome message.

图 1: GradeBook 类的 UML 类图，由 doxygen 生成



目录

面向对象编程

类的定义与使用

成员函数

数据成员

构造函数

多文件





程序二：具有形参的成员函数

```
1  /** @file fig03_03.cpp
2   ** Define a class with a member function that takes a parameter
3   */
4  #include <iostream>
5  #include <string> // C++ standard string class
6
7  /** @class GradeBook
8   ** @brief First definition of class */
9  class GradeBook {
10 public:
11     /** @brief Display Welcome message */
12     void displayMessage(std::string courseName) const {
13         std::cout << "Welcome to the grade book for\n" << courseName << "!" << std::endl;
14     } // end function
15 }; // end class
16
17 int main() {
18     std::string nameOfCourse; // string of characters to store the course name
19     GradeBook myGradeBook; // create a GradeBook object named myGradeBook
20
21     std::cout << "Please enter the course name: " << std::endl;
22     std::getline(std::cin, nameOfCourse); // read a course name with blanks
23
24     std::cout << std::endl;
25     myGradeBook.displayMessage(nameOfCourse); // call object's member function with argument
26 }
```



Line 12-14: 具有形参的函数

```
11  /** @brief Display Welcome message */
12  void displayMessage(std::string courseName) const {
13      std::cout << "Welcome to the grade book for\n" << courseName << "!" << std::endl;
14  } // end function
```

回顾: C++ 函数

返回类型 函数名(输入参数) { 函数体; }

括号 () 内的输入参数列表 (parameter list) 按类型 参数名给出

- 无参数时: () 或 (void)
- 1 个参数: (std::string courseName)
- 多个参数²: (std::string courseName, int x, int y)

²逗号隔开



Line 25: 带实参的函数调用

```
11 class GradeBook {
12 public:
13     /** @brief Display Welcome message */
14     void displayMessage(std::string courseName) const {
15         std::cout << "Welcome to the grade book for\n" << courseName << "!" << std::endl;
16     } // end function
17 }; // end class

25 myGradeBook.displayMessage(nameOfCourse); // call object's member function with argument
```

形参与实参

函数定义内的输入参数又称形式参数(形参/parameter), 调用该函数时必须传入数量、类型、顺序一致的实际参数(实参/argument)

- 多个实参同样用逗号分隔, 不写类型
- 形参的名字是个代号, 表示会对传进来的数做什么, 如

```
int minus(int x, int y) { return x - y; }
```

- 实参是实际执行时的值, 可以是变量、常量、表达式, 如
minus(b * b, 4 * a * c)



Line 18: std::string

<https://cplusplus.com/reference/string/string/>

```
5  #include <string>

17 int main() {
18     std::string nameOfCourse;
```

std::string是 C++ 标准库为字符串定义的类。因此我们可以创建类型为std::string的对象nameOfCourse。

- 注意std::string不是基本类型，它定义在 std 命名空间下，因此需要#include <string>，并且可以通过using声明简写
- 可将字符串赋值给std::string对象，如nameOfCourse = "C++";
- std::string类同样有自己的成员函数，例如nameOfCourse.size()会返回字符串中的字符(char)数



Line 22: std::getline

<https://cplusplus.com/reference/string/string/getline/>

22

```
std::getline(std::cin, nameOfCourse); // read a course name with blanks
```

std::getline 函数声明

```
istream& getline (istream& is, string& str, char delim);  
istream& getline (istream& is, string& str);
```

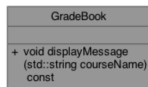
- 从输入流is将字符读入str，直到遇到终止符delim或换行符'\n'
 - 从标准输入流std::cin将字符读入nameOfCourse，直到遇到换行符
- 和std::cin >> nameOfCourse;的区别是，运算符>>遇到空白字符（包括' ', '\n', '\t'）就停
- std::getline不会把换行符放进str，但会放进空格

更新后的 UML 类图

GradeBook Class Reference

First definition of class. [More...](#)

Collaboration diagram for GradeBook:



Public Member Functions

```
void displayMessage(std::string courseName) const
```

Display Welcome message.

图 2: GradeBook 类更新后的 UML 类图，由 doxygen 生成

由于 UML 是跨语言的，可能会有不同格式或术语，如 `displayMessage(courseName : String)`



练习：用 C++ 创建类、对象和成员函数

小练习

请尝试写一个Calculator类，包含 computeCircumference 和 computeArea 两个函数，输入圆半径，分别计算周长和面积。



练习：用 C++ 创建类、对象和成员函数

小练习

请尝试写一个 Calculator 类，包含 computeCircumference 和 computeArea 两个函数，输入圆半径，分别计算周长和面积。

```
1 // example.cpp
2 #include <iostream>
3
4 constexpr double PI = 3.14159;
5
6 class Calculator {
7 public:
8     double computeCircumference(double radius) const {
9         return 2 * PI * radius;
10    }
11    double computeArea(double radius) const {
12        return PI * radius * radius;
13    }
14 };
15
16 int main () {
17     double radius;
18     std::cout << "Please enter the radius: ";
19     std::cin >> radius;
20     Calculator calculator;
21     std::cout << "Circumference = " << calculator.computeCircumference(radius) << ", "
22               << "Area = " << calculator.computeArea(radius) << std::endl;
23 }
```



目录

面向对象编程

类的定义与使用

成员函数

数据成员

构造函数

多文件





程序三：具有数据成员类

```
1  /** @file fig03_05.cpp
2   ** Define a class containing a data member with setter/getter */
3  #include <iostream>
4  #include <string> // C++ standard string class
5  /** @class GradeBook
6   ** @brief First definition of class */
7  class GradeBook {
8  public:
9      /** @brief Setter */
10     void setCourseName(std::string name) { courseName = name; }
11     /** @brief Getter */
12     std::string getCourseName() const { return courseName; }
13     /** @brief Display Welcome message */
14     void displayMessage() const {
15         std::cout << "Welcome to the grade book for\n" << getCourseName() << "!" << std::endl;
16     }
17 private:
18     std::string courseName;
19 }; // end class
20 int main() {
21     std::string nameOfCourse; // string of characters to store the course name
22     GradeBook myGradeBook; // create a GradeBook object named myGradeBook
23     std::cout << "Initial course name is: " << myGradeBook.getCourseName() << std::endl; // display initial value
24     std::cout << "\nPlease enter the course name: " << std::endl;
25     std::getline(std::cin, nameOfCourse); // read a course name with blanks
26     myGradeBook.setCourseName(nameOfCourse); // set the course name
27     std::cout << std::endl;
28     myGradeBook.displayMessage();
29 }
```



Line 17-18: 数据成员

https://en.cppreference.com/cpp/language/data_members

```
17 private:  
18     std::string courseName;
```

回顾：在 C++ 中声明变量

数据类型 变量名, 变量名;

数据类型 变量名 = 初始值, 变量名 = 初始值;

- 类中的变量称为**数据成员** (data member)
- 定义在某个函数大括号内的变量称为**局部变量** (local variable)
- 定义在所有大括号外部的变量称为**全局变量** (global variable)
- 数据成员用于记录对象的属性, 每个对象都有自己的数据成员值



数据成员、局部变量与全局变量（拓展）

全局变量在任何地方都可见

```
#include <iostream>
```

```
int x = 1; // global variable
```

```
class MyClass {  
public:  
    void display() const {  
        std::cout << x << std::endl;  
    }  
};
```

```
int main() {  
    x = x + 1; // x == 2  
    MyClass obj;  
    obj.display(); // display global x == 2  
}
```



数据成员、局部变量与全局变量（拓展）

局部变量在自己所属的大括号内可见

```
#include <iostream>
```

```
int main() {  
    {  
        int y = x; // error: 'x' was not declared  
        int x = 1; // local variable within inner bracket  
        x = x + 1;  
        {  
            std::cout << x << std::endl; // display x == 2  
        }  
    }  
    x = x + 1; // error: 'x' was not declared in this scope  
}
```

变量可见范围称为其作用域 (scope)



数据成员、局部变量与全局变量（拓展）

数据成员在类内可见³，类外通过对象名．数据成员访问

```
#include <iostream>
```

```
class MyClass {  
public:  
    void display() const {  
        std::cout << x << std::endl;  
    }  
    int x = 1;  
};  
  
int main() {  
    MyClass obj1, obj2;  
    obj1.x = obj1.x + 1; // obj1.x == 2, obj2.x == 1  
    obj1.display(); // display obj1.x == 2  
    std::cout << obj2.x << std::endl; // display obj2.x == 1  
}
```

³编译器先处理所有成员再处理函数体，因此允许数据成员定义在成员函数使用后



访问说明符

<https://en.cppreference.com/cpp/language/access>

我们已经学过声明为`public`的成员函数，可通过`obj.func()`调用

- 声明为`public`的数据成员，也可通过`obj.var`指定

```
#include <iostream>

class MyClass {
public:
    void display() const {
        std::cout << x << std::endl;
    }
    int x = 1;
};

int main() {
    MyClass obj;
    obj.x = obj.x + 1;
    obj.display(); // x == 2
}
```

- 声明为`private`的成员，仅能被类内部成员访问

```
#include <iostream>

class MyClass {
private:
    void display() const {
        // within class, no problem
        std::cout << x << std::endl;
    }
    int x = 1;
};

int main() {
    MyClass obj;
    // error: 'x' is private
    obj.x = obj.x + 1;
    // error: display() is private
    obj.display();
}
```



私有访问与封装（拓展）

- 理论上，把任何`private`改成`public`都不影响代码编译执行
- 既然如此，为什么还需要`private`私有访问？

封装 (Encapsulation)

将相关数据和操作数据的函数组织到一个类中，并隐藏细节，只通过接口交互

- 数据隐藏 (Data Hiding): 阻止外部代码直接访问数据，避免误用
- 数据抽象 (Data Abstraction): 对外只暴露接口，不暴露具体实现
- `private`的数据成员仅能由类内部访问，外部如何使用？



Line 9-12: Setter 与 Getter

```
9  public:
10     /** @brief Setter */
11     void setCourseName(std::string name) { courseName = name; }
12     /** @brief Getter */
13     std::string getCourseName() const { return courseName; }
```

- 通过 Setter 给数据成员赋值，通过 Getter 得到数据成员的当前值
- 由于成员 `courseName` 是 `private`，我们无需担心外部代码用 `setCourseName(name)` 以外的方式给它赋值
 - 有时也会将 Setter 设置为 `private`，完全避免从外部修改
- 若数据成员的值不符合预期，只需要检查类内部及调用 Setter 的地方⁴，而无需检查全部用到该成员的地方
 - 大项目中，读取值的次数 (get) 要远多于赋值次数 (set)
- `const` 关键字写在函数体前，承诺该函数不会修改任何数据成员
 - 因此，Setter 不会是 `const` 的，Getter 通常是 `const` 的

⁴关于类内部是直接访问数据成员，还是使用 Setter/Getter，存在风格争议。



用 Setter 确认数据有效性

Setter 也用于检查数据合法性⁵，如限制courseName最多 25 个字符

std::string 部分成员函数

```
size_t size() const;
```

```
size_t length() const;
```

二者等价，返回字符串长度（size_t等同于unsigned int）

```
string substr (size_t pos = 0, size_t len = npos) const;
```

返回从 pos 位置开始（首位为 0），最多 len 个字符的子字符串

```
void setCourseName(std::string name) {  
    if (name.size() <= 25) courseName = name;  
    if (name.size() > 25) {  
        courseName = name.substr(0, 25); // set to the first 25 characters  
        std::cerr << "Name \"<\" << name << "\" exceeds maximum length (25).\\n"  
                    "Limited to first 25 characters.\\n" << std::endl; // merge strings  
    }  
}
```

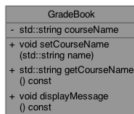
⁵原则上，Setter 不应包含太过复杂的验证 (Validation) 逻辑，以确保效率

含 Setter 与 Getter 的 UML 类图

GradeBook Class Reference

First definition of class. [More...](#)

Collaboration diagram for GradeBook:



Public Member Functions

```
void setCourseName (std::string name)
```

Setter.

```
std::string getCourseName () const
```

Getter.

```
void displayMessage () const
```

Display Welcome message.

Private Attributes

```
std::string courseName
```

图 3: 含 Setter 与 Getter 的 GradeBook 类 UML 类图, 由 doxygen 生成



目录

面向对象编程

类的定义与使用

成员函数

数据成员

构造函数

多文件





std::string 的初始化

<https://cplusplus.com/reference/string/string/string/>

程序三中还有个知识点

```
22 GradeBook myGradeBook; // create a GradeBook object named myGradeBook
23 std::cout << "Initial course name is: " << myGradeBook.getCourseName() << std::endl; // display initial value
```

- 在通过setCourseName(nameOfCourse)给courseName赋值之前，我们先直接调用了getCourseName()，得到了空字符串""
- 但基础类型（如int）声明时若不赋初始，其中是垃圾值⁶

```
#include <iostream>
class MyClass {
public:
    int getValue() const { return value; }
private:
    int value; // uninitialized
};

int main() {
    MyClass obj;
    std::cout << obj.getValue() << std::endl; // undefined behavior
}
```

- 基础类型变量不会自动初始化，对象创建会调用类的构造函数
 - 当通过std::string str;构建一个对象时，调用std::string的默认构造函数，将它初始化为空字符串

⁶ 严格来说，读取未初始化的变量属于 Undefined Behavior，取决于编译器的具体实现



程序四：具有显式构造函数的类

```
1  /** @file fig03_07.cpp
2  ** Define a class with explicit constructor */
3  #include <iostream>
4  #include <string> // C++ standard string class
5  /** @class GradeBook
6  ** @brief First definition of class */
7  class GradeBook {
8  public:
9      GradeBook(std::string name): courseName(name) { }
10     /** @brief Setter */
11     void setCourseName(std::string name) { courseName = name; }
12     /** @brief Getter */
13     std::string getCourseName() const { return courseName; }
14     /** @brief Display Welcome message */
15     void displayMessage() const {
16         std::cout << "Welcome to the grade book for\n" << getCourseName() << "!" << std::endl;
17     }
18 private:
19     std::string courseName;
20 }; // end class
21 int main() {
22     // create two objects
23     GradeBook gradeBook1("CS101 Introduction to C++ Programming");
24     GradeBook gradeBook2{"CS102 Data Structures in C++"};
25     // display initial value of courseName
26     std::cout << "gradeBook1 created for course: " << gradeBook1.getCourseName()
27         << "\ngradeBook2 created for course: " << gradeBook2.getCourseName() << std::endl;
28 }
```



Line 9: 构造函数

```
7  class GradeBook {  
8      public:  
9          GradeBook(std::string name): courseName(name) { }
```

C++ 类的构造函数

类名 (输入参数):成员初始化列表 {
 函数体;
}

- 构造函数 (Constructor) 是一种特殊的成员函数
 - 没有返回类型 (不是 `void`), 函数名与类名一致
 - 可以没有参数, 也可以有多个参数, 括号不能省略
 - 若成员初始化列表 (member initializer list) 为空, 则冒号也不写
 - 成员初始化列表格式为数据成员 (初始值), 数据成员 (初始值)
- 每次构建对象, 都会调用该类的构造函数
 - 执行顺序为先按数据成员定义顺序, 依次根据成员初始化列表进行初始化 (与列表顺序无关), 再执行函数体



构造函数执行顺序（拓展）

```
class MyClass {  
public:  
    MyClass(int a): y(x + 1), x(a) {  
        z = x + y;  
    }  
private:  
    int x = 0;  
    int y;  
    int z;  
};  
  
int main() {  
    MyClass obj(1); // x == 1, y == 2, z == 3  
}
```

- ① 初始化`int x`;根据列表, 将`x`初始化为`a == 1`
- ② 初始化`int y`;根据列表, 将`y`初始化为`x + 1 == 2`
- ③ 初始化`int z`;由于`z`不在列表中, `int`为基础类型, 此时`z`未初始化
- ④ 执行函数体, `z`的值变为 3
 - 由于变量值发生改变, 常量`const int`无法在函数体内初始化



初始化优先级（拓展）

https://en.cppreference.com/cpp/language/default_initialization

- **成员初始化列表** (member initializer list) 拥有最高优先级，其中出现的数据成员一定按列表值进行初始化
- 定义数据成员时给定的初始值用于**默认成员初始化** (default member initializer)，若不在成员初始化列表中，则按这个值进行
- 若都没有，则进行默认初始化 (default initialization)
 - 基础类型成员不自动初始化（可能为垃圾值）
 - 类成员调用默认构造函数，如`std::string`初始化为""

```
class MyClass {  
    const int x{1};  
    int y{2}, z;  
    std::string s;  
public:  
    MyClass(): x{3} { }  
};
```

- 我们在第43页介绍，可用大括号进行初始化
- `x == 3`: 成员初始化列表优先级最高
- `y == 2`: 默认成员初始化
- `z == ?`: 未初始化
- `s == ""`: 默认构造函数



默认构造函数

https://en.cppreference.com/cpp/language/default_constructor

- 没有输入参数的构造函数称为默认构造函数 (default constructor)
- 若没有为类声明构造函数，则 C++ 会隐式声明一个默认构造函数，其访问权限为 **public**
 - 一旦用户定义任何构造函数（不论是否有参数），自动隐式默认构造函数都会被删除

- MyClass类的隐式默认构造函数等价于如下构造函数

```
class MyClass {  
public:  
    MyClass() {}  
};
```

- 用户也可以重新定义默认构造函数，如

```
MyClass() { std::cout << "User-defined default constructor." << std::endl; }
```

- 如果想保留默认构造函数，可以简写为 `MyClass() = default;`
- 如果想删除默认构造函数，可使用 `MyClass() = delete;`
- 如果想禁止外部构造，也可定义 **private** 的构造函数



Line 23-24: 通过构造函数构造对象

```
21 int main() {  
22     // create two objects  
23     GradeBook gradeBook1("CS101 Introduction to C++ Programming");  
24     GradeBook gradeBook2{"CS102 Data Structures in C++"};
```

- 有形参的构造函数，可以使用 类名 对象名(实参列表)；或者 类名 对象名 {实参列表}；进行构造
- 无形参的构造函数，可以使用 类名 对象名；或者 类名 对象名 {}；进行构造
 - 即MyClass obj;或MyClass obj{};
 - 不能用圆括号！
- 作为返回值等匿名的场景，直接用当作函数，用函数名调用⁷
 - return MyClass();
 - std::cout << GradeBook("test").getCourseName();

⁷此时可以用圆括号也可用大括号



Vexing parse (拓展)

为什么无参数的类对象初始化不能用圆括号？

C+ 最令人烦恼的解析 (most vexing parse)

MyClass obj();

- 解释一：类名 **对象名** (空的实参列表); 因此这是对象的初始化
- 解释二：返回类型 **函数名** (空的形参列表); 因此这是个返回类型为 MyClass 的函数

编译器往往默认采用解释二 (可用开关控制)，因此如下代码可编译

```
#include <iostream>
class MyClass {
public:
    MyClass() { std::cout << "Default Constructor of MyClass." << std::endl; }
};

MyClass func();
MyClass func() {
    std::cout << "A function that returns MyClass type." << std::endl;
    return MyClass();
}

MyClass obj; // output line 1: Default Constructor

int main() {
    func(); // output line 2-3: A function + Default Constructor
}
```



统一初始化（拓展）

https://en.cppreference.com/cpp/language/list_initialization

现代 C++（11 起）推荐使用大括号来统一初始化 (Uniform initialization)

统一初始化

- 既可用于基础类型，也可用于类与对象，有无参数都可用

```
int x{0};  
MyClass obj{};
```

- 既可用于局部变量，也可用于数据成员（含成员初始化列表）

```
class MyClass {  
    int x, y{};  
public:  
    MyClass(): x{} { }  
};
```

- 禁止有损类型转换

```
int x = 1.9; // OK, x == 1  
int y{1.9}; // error: narrowing conversion
```

- 对基础数据类型，空列表 {} 会进行零初始化 (zero-initialization)^a

```
int x{}; // x == 0  
bool b{}; // b == false
```

^ahttps://en.cppreference.com/cpp/language/zero_initialization



explicit 关键字

<https://en.cppreference.com/cpp/language/explicit>

由于 C++ 支持隐式类型转换，可能会出现如下比较费解的代码

```
class MyClass {  
    int value;  
public:  
    MyClass(int v): value(v) { }  
};  
  
int main() {  
    MyClass obj{1}; obj = 2;  
}
```

看似把整数 2 直接赋值给 obj，但其实 C++ 利用构造函数将 int “转化”为了 MyClass 类型。在大项目中，这种隐式转换非常混乱，因此我们用 explicit 关键词，要求构造函数被调用时名字必须出现。

```
explicit MyClass(int v): value(v) { }
```

如此修饰后，之前的命令就无法编译了，需要 obj = MyClass{2};

这种代码风格更清晰，推荐构造函数都加上 explicit

含构造函数的 UML 类图

GradeBook Class Reference

First definition of class. [More...](#)

Collaboration diagram for GradeBook:



Public Member Functions

`GradeBook (std::string name)`

`void setCourseName (std::string name)`

Setter.

`std::string getCourseName () const`

Getter.

`void displayMessage () const`

Display Welcome message.

Private Attributes

`std::string courseName`

图 4: 含构造函数的 GradeBook 类 UML 类图, 由 doxygen 生成



练习：类的构造函数与数据成员

回顾

请尝试写一个Calculator类，包含 `computeCircumference` 和 `computeArea` 两个函数，输入圆半径，分别计算周长和面积。

小练习

请修改Calculator类，使得`radius`是数据成员，并可在构造时传入。



练习：类的构造函数与数据成员

小练习

请修改Calculator类，使得radius是数据成员，并可在构造时传入。

```
1 // example.cpp
2 #include <iostream>
3
4 constexpr double PI = 3.14159;
5
6 class Calculator {
7     double radius;
8 public:
9     explicit Calculator(double r): radius(r) { }
10    void setRadius(double r) { radius = r; }
11    double getRadius() const { return radius; }
12    double computeCircumference() const { return 2 * PI * radius; }
13    double computeArea() const { return PI * radius * radius; }
14 };
15
16 int main () {
17     double radius;
18     std::cout << "Please enter the radius: ";
19     std::cin >> radius;
20     Calculator calculator{radius};
21     std::cout << "Circumference = " << calculator.computeCircumference() << ", "
22               << "Area = " << calculator.computeArea() << std::endl;
23 }
```



目录

面向对象编程

类的定义与使用

成员函数

数据成员

构造函数

多文件





为什么需要多个文件

随着程序规模增大，把所有代码写在一个源文件中会越来越困难

- 一个程序可能包含几十甚至几百个类
- 不同开发者需要同时开发不同模块
- 修改一个类时，不希望影响其他代码
- 希望已经写好的代码能够在其他程序中重复使用

代码复用 (Code Reuse)

多次使用同一段代码，避免“重复造轮子”

数据抽象 (Abstraction)

将对象的使用方式与实现细节分离，仅通过对外接口交互

简化代码，降低不同模块之间的耦合，方便维护，容易复用

程序五：将类放入单独的头文件

头文件 (Header File)

```

1  /** @file GradeBook.h
2  ** GradeBook header file */
3  #include <iostream>
4  #include <string> // C++ standard string class
5  /** @class GradeBook
6  ** @brief First definition of class */
7  class GradeBook {
8  public:
9      explicit GradeBook(std::string name)
10         : courseName(name) { }
11      /** @brief Setter */
12      void setCourseName(std::string name) {
13          courseName = name;
14      }
15      /** @brief Getter */
16      std::string getCourseName() const {
17          return courseName;
18      }
19      /** @brief Display Welcome message */
20      void displayMessage() const {
21          std::cout << "Welcome to the grade book for\n"
22                  << getCourseName() << "!" << std::endl;
23      }
24  private:
25      std::string courseName;
26  }; // end class
  
```

源文件 (Source File)

```

1  /** @file fig03_10.cpp
2  ** main file */
3  #include <iostream>
4  #include "GradeBook.h"
5
6  int main() {
7      // create two objects
8      GradeBook book1("CS101 Introduction to C++ Programming");
9      GradeBook book2{"CS102 Data Structures in C++"};
10     // display initial value of courseName
11     std::cout << "book1 created for course: "
12             << book1.getCourseName()
13             << "\nbook2 created for course: "
14             << book2.getCourseName() << std::endl;
15 }
  
```



头文件

表 2: 头文件与源文件对比

	头文件	源文件
后缀名 ⁸	.h, .hpp	.cpp, .c, .cxx
内容	类定义、函数声明	函数定义
使用	引用 <code>#include ".h"</code>	编译 <code>g++ -c .cpp</code>

- 我们提到过, `#include` 可理解为文本替换, 替换后与程序四一致
- 多次引用同一个头文件可能导致类重复定义

```
#include <iostream>
#include "GradeBook.h"
#include "GradeBook.h" // error: redefinition of 'class GradeBook'
```

- 为防止这种现象, 应养成习惯在头文件开头写 `#pragma once` 预处理指令, 或在首尾定义防护符

```
#ifndef GRADEBOOK_H
#define GRADEBOOK_H
...
#endif // GRADEBOOK_H
```

- 头文件也可引用其他头文件, 原则是用到什么就引用什么

⁸C++ 源代码均为文本文件, 编译器不根据后缀名区分文件内容, 仅作参考提示



头文件搜索与编译

<https://en.cppreference.com/cpp/preprocessor/include>

- 头文件无需单独编译，我们目前的编译指令依然是g++ main.cpp
- 编译器预处理#include指令时，会根据引用方式搜索文件
 - 如果使用双引号，优先搜索源文件所在目录（不是执行g++的目录）
 - 如果没找到，或使用尖括号，则按以下顺序搜索
 - 编译指令中-I <path>指定的目录
 - 标准库目录
 - 尖括号不会搜索源文件目录，因此用于引用<标准库头文件>；双引号用于引用“自定义头文件”
- 头文件中不应出现using指令（尽管这不是个编译错误）
 - 这么做会污染命名空间，任何使用该头文件的用户都必须接受string指代std::string
 - 根据最小引用原则，即使在源文件中，最好也分别引用，避免using namespace std;



分离接口与实现

- 通过分离类的定义与main函数，我们的GradeBook类现在可被复用
- 但GradeBook.h文件依然同时包含接口与实现，会导致一些问题
 - 用户使用GradeBook类时可能过度依赖实现而非接口
 - 一旦修改实现，任何引用GradeBook.h的源文件都必须重新编译
- 我们希望头文件GradeBook.h仅保留最少的必要说明
 - 让编译器知道如何编译
 - 让用户知道如何使用
- 将具体的实现细节放到GradeBook.cpp中，用户无需关注

接口 (Interface)

接口是一种契约 (promise)，只规定类可以提供哪些功能而不暴露这些功能如何实现



程序六：分离接口与实现

头文件 (GradeBook.h)

```

1  /** @file GradeBook.h
2  ** GradeBook header file */
3  #ifndef GRADE_BOOK_H
4  #define GRADE_BOOK_H
5  #include <string> // C++ standard string class
6  /** @class GradeBook
7  ** @brief First definition of class */
8  class GradeBook {
9  public:
10     explicit GradeBook(std::string);
11     /** @brief Setter */
12     void setCourseName(std::string);
13     /** @brief Getter */
14     std::string getCourseName() const;
15     /** @brief Display Welcome message */
16     void displayMessage() const;
17 private:
18     std::string courseName;
19 }; // end class
20 #endif

```

实现源文件 (GradeBook.cpp)

```

1  /** @file GradeBook.cpp
2  ** GradeBook function definitions */
3  #include "GradeBook.h"
4
5  #include <iostream>
6
7  GradeBook::GradeBook(std::string name)
8      : courseName(name) { }
9
10 void GradeBook::setCourseName(std::string name) {
11     courseName = name;
12 }
13
14 std::string GradeBook::getCourseName() const {
15     return courseName;
16 }
17
18 void GradeBook::displayMessage() const {
19     std::cout << "Welcome to the grade book for\n"
20               << getCourseName() << "!" << std::endl;
21 }

```



GradeBook.h: 接口

```
8  class GradeBook {
9  public:
10     explicit GradeBook(std::string);
11     /** @brief Setter */
12     void setCourseName(std::string);
13     /** @brief Getter */
14     std::string getCourseName() const;
15     /** @brief Display Welcome message */
16     void displayMessage() const;
17 private:
18     std::string courseName;
19 }; // end class
```

- 接口中保留**数据成员** + **成员函数的函数原型** (function prototype)
- 数据成员
 - 每个对象持有数据成员，编译器需要知道为每个对象分配多少空间
 - 用户可能需要通过点运算符 (.) 访问公开数据成员
- 函数原型
 - 函数原型是一种函数声明 (declaration)
 - 必备信息是返回类型 + 函数名 + 形参类型 + 修饰符，形参名可选
 - 有这些信息，编译器和用户就知道如何使用该函数了



函数声明与定义

```
#include <iostream>
// declaration + definition
int foo(int x, int y) { return x + y; }

// forward declaration
int bar(int a, int b);

int main() {
    std::cout << foo(1,1) << bar(1,1) << std::endl;
}

// definition
int bar(int x, int y) { return x - y; }
```

- 函数声明 (Declaration) 表示函数该怎么用, 定义 (Definition) 表示具体怎么实现
- 一般而言, 声明在头文件中, 定义在源文件中 (也有例外)
- 可以在定义的同时声明, 也可以先声明后定义
- 声明可以有很多次, 定义只能有一个
- 声明用分号 (不能是空大括号!), 定义用大括号



GradeBook.cpp: 实现

```
3 #include "GradeBook.h"
4
5 #include <iostream>
6
7 GradeBook::GradeBook(std::string name)
8     : courseName(name) { }
```

- 一般而言，我们将.cpp 文件对应的.h 文件放在 include 最前面，后面依次是标准库、其他头文件
- 由于已经定义过GradeBook类，此处不能重新定义，只要给出此前声明过的函数的具体实现
- **operator::**为作用域分辨运算符，在 C++ 运算符中优先级最高
 - ::前为类名或命名空间，如std::cout
 - 注意与对象名、成员区分，如str.size()
- 此处需要GradeBook::来标明它们是 GradeBook 类的成员函数，否则编译器会理解为该.cpp 文件内的全局函数



多文件编译

- 我们现在有多个.cpp 文件，其中只有一个有main函数，是程序的主入口
- 直接使用g++ main.cpp编译，会提示找不到函数定义

多文件编译

- ① 通过g++ -c file.cpp -o file.o将源文件分别编译为目标文件
 - ② 通过g++ file.o ... -o main将目标文件链接为可执行程序
- 用户只接触.o 目标文件（机器指令），而未接触源代码，实现隔离
 - 链接阶段，链接器将接口与目标文件中的实现代码链接
 - 整个程序中应当有且只有一个 main 函数



Makefile (拓展)

现在 C++ 工程通常用 CMake，用 Makefile 可以更直观理解编译过程。

```
1 all: main
2
3 main: main.o GradeBook.o
4     g++ $^ -o $@
5
6 %.o: %.cpp
7     g++ -c $< -o $@
```

- 在目录下创建一个Makefile文件，即可通过make指令一键编译
- `all: main`, make默认执行第一个目标，通过 `all` 允许多目标
- `main: main.o GradeBook.o`, 程序 `main` 依赖两个目标文件
 - 编译方式为 `g++` 依赖文件 `-o` 目标
- `%.o: %.cpp`, 每个 `.o` 文件依赖对应的 `.cpp` 文件
 - 编译方式为 `g++ -c` 依赖文件 `-o` 目标.o
- `make`可根据文件保存时间，避免重复编译无改动的文件

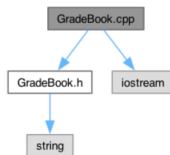
文件依赖关系图

GradeBook.cpp File Reference

```
#include "GradeBook.h"
```

```
#include <iostream>
```

Include dependency graph for GradeBook.cpp:



Detailed Description

GradeBook function definitions

图 5: 文件依赖关系图, 由 doxygen 生成

由于 GradeBook.h 中没有用到 <iostream> 的成员, 因此不引用比较清晰



练习：类的构造函数与数据成员

小练习

将 Calculator 类拆分为多文件



总结

- 理解面向对象的基本概念✓
 - 类、对象、数据成员、成员函数、封装、数据隐藏、数据抽象
- 掌握创建类，创建、使用和删除类对象的原理和方法✓
 - 创建类：`class MyClass { };`
 - 创建对象：`MyClass obj{}`;
 - 使用对象：`obj.func()`;
 - 删除将在后续章节中介绍
- 掌握控制对象数据成员和成员函数的访问✓
 - `public`, `private`
 - `obj.setValue(v)`;
 - `obj.getValue()`;
- 掌握如何将接口同实现分离✓
 - 头文件、源文件、接口
- 用构造函数初始化对象✓
 - 定义：`MyClass(int v): value{v} { }`
 - 使用：`MyClass obj{1}`;